

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 4

Comparative Study of Deep Convolutional Neural Network Architectures Using Transfer Learning

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

The objectives of this experiment are

- Study the evolution of deep CNN architectures.
- Compare LeNet-5, AlexNet, VGG16, GoogleNet and ResNet.
- Understand transfer learning.
- Fine tune pretrained CNN models.
- Compare classification performance of different architectures.

2. Learning Outcomes

After completing this experiment students will be able to

1. Explain modern CNN architectures.
2. Differentiate LeNet, AlexNet, VGG16, GoogleNet and ResNet.
3. Implement transfer learning.
4. Fine tune pretrained CNN models.
5. Compare CNN architectures using performance metrics.

3. Introduction

Deep Convolutional Neural Networks have become the dominant models for computer vision applications because they automatically learn hierarchical features from images. The evolution of CNNs has focused on improving classification accuracy while reducing computational complexity and training difficulties.

The progression of CNN architectures began with LeNet-5 and has evolved through AlexNet, VGGNet, GoogleNet, and ResNet. Each architecture introduced important innovations that significantly improved image classification performance.

4. Evolution of CNN Architectures

Model	Year	Major Contribution
LeNet-5	1998	First practical convolutional neural network
AlexNet	2012	ReLU activation, Dropout and GPU training
VGG16	2014	Uniform 3×3 convolution filters
GoogleNet	2014	Inception modules for multi-scale feature extraction
ResNet	2015	Residual learning using skip connections

5. LeNet-5

LeNet-5 was proposed by Yann LeCun in 1998 for handwritten digit recognition. It consists of two convolution layers, two average pooling layers and fully connected layers. It demonstrated that convolutional networks could automatically learn image features without handcrafted feature extraction.

Advantages

- Small number of parameters
- Fast training
- Suitable for OCR

6. AlexNet

AlexNet won the ImageNet Challenge in 2012 by reducing the classification error significantly. It introduced ReLU activation, Dropout regularization and GPU training.

Major innovations

- ReLU activation
- Dropout
- GPU implementation
- Data augmentation

7. VGG16

VGG16 demonstrated that using multiple small 3×3 convolution filters could outperform larger filters while increasing network depth.

Advantages

- Excellent feature extraction
- Simple architecture
- High classification accuracy

8. Comparison of Architectures

Architecture	Depth	Parameters	Main Contribution
LeNet-5	7	60K	First CNN
AlexNet	8	61M	ReLU + Dropout
VGG16	16	138M	Deep 3×3 filters
GoogLeNet	22	6.8M	Inception Modules
ResNet50	50	25.6M	Residual Learning

9. GoogLeNet (Inception Network)

GoogLeNet, proposed by Szegedy *et al.* in 2014, introduced the **Inception Module**, which performs multiple convolution operations in parallel using different filter sizes. Instead of using only a single kernel size, GoogLeNet simultaneously applies 1×1 , 3×3 , 5×5 convolutions and max pooling. The outputs are concatenated to obtain multi-scale feature representations.

The use of 1×1 convolutions before larger kernels reduces the number of trainable parameters and computational complexity.

Advantages

- Multi-scale feature extraction.
- Fewer parameters than VGG16.
- High computational efficiency.
- Better classification performance.

10. ResNet

Increasing CNN depth often causes the **vanishing gradient problem**, making optimization difficult. ResNet solves this issue using **skip (residual) connections**, allowing gradients to flow directly through the network.

Instead of learning

$$H(x)$$

ResNet learns

$$F(x) = H(x) - x$$

thus,

$$Output = F(x) + x$$

where

- $F(x)$ = Residual Mapping
- x = Identity Mapping

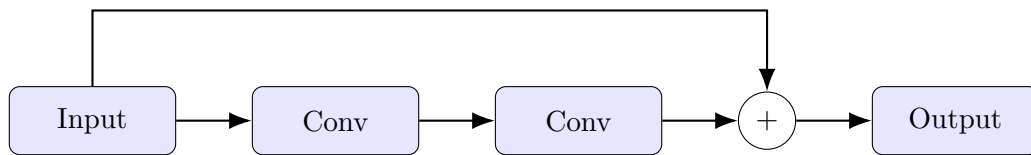


Figure 1: Residual Block in ResNet

Advantages

- Eliminates vanishing gradients.
- Enables very deep neural networks.
- Faster convergence.
- High ImageNet accuracy.

11. Dilated Convolution

Dilated convolution (also called atrous convolution) increases the receptive field without increasing the number of parameters.

The dilation rate determines the spacing between kernel elements.

For a standard convolution,

$$D = 1$$

For dilated convolution,

$$D > 1$$

Example

$$\begin{bmatrix} 1 & 0 & 2 \\ 0 & 3 & 0 \\ 4 & 0 & 5 \end{bmatrix}$$

where zeros indicate inserted gaps.

Applications

- Semantic segmentation
- Medical image analysis

- Satellite imagery
- Object localization

12. Transpose Convolution

Transpose convolution increases the spatial dimensions of feature maps. Unlike pooling, which reduces image size, transpose convolution performs learnable upsampling.

Applications

- Image Super Resolution
- Autoencoders
- GANs
- Image Generation
- Semantic Segmentation

13. Transfer Learning

Transfer Learning utilizes a pretrained CNN model that has already learned general image features from a large dataset such as ImageNet.

Instead of training from scratch,

1. Load a pretrained model.
2. Remove the original classifier.
3. Freeze convolution layers.
4. Add new Dense layers.
5. Train the classifier.
6. Fine tune selected convolution layers.

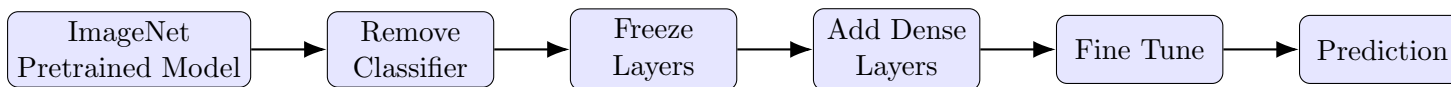


Figure 2: Transfer Learning Workflow

Advantages

- Faster convergence.
- Higher accuracy on small datasets.
- Reduced training time.
- Lower computational cost.

14. Dataset

Dataset: CIFAR-10

- Training Images : 50,000
- Testing Images : 10,000
- Number of Classes : 10
- Image Size : $32 \times 32 \times 3$
- Classes: Airplane, Automobile, Bird, Cat, Deer, Dog, Frog, Horse, Ship and Truck.

15. Experimental Procedure

Task 1: Dataset Preparation

1. Load the CIFAR-10 dataset using TensorFlow/Keras.
2. Normalize the pixel values to the range $[0,1]$.
3. Display ten sample images.

4. Print the dimensions of the training and testing datasets.

Task 2: Transfer Learning

Load any one of the following pretrained CNN models.

- VGG16
- ResNet50
- MobileNetV2
- EfficientNetB0 (Optional)

Perform the following steps.

1. Load pretrained ImageNet weights.
2. Remove the original classification layer.
3. Freeze the convolutional base.
4. Add a Global Average Pooling layer.
5. Add a Dense layer with ReLU activation.
6. Add the output layer with Softmax activation.

Task 3: Model Training

Train the model using

Optimizer	Adam
Learning Rate	0.001
Batch Size	32
Epochs	10–20
Loss Function	Categorical Cross Entropy
Evaluation Metric	Accuracy

Task 4: Fine Tuning

1. Unfreeze the last convolution block.
2. Train for another 5–10 epochs.
3. Compare the classification accuracy before and after fine tuning.

Task 5: Model Evaluation

Evaluate the trained model using

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

16. Hyperparameter Study

Compare the performance using different settings.

Hyperparameter	Values
Learning Rate	0.001, 0.0001
Batch Size	16, 32, 64
Epochs	10, 20
Optimizer	Adam, SGD
Dense Units	128, 256
Frozen Layers	All, Partial

17. Mandatory Plots



Figure 3: Sample CIFAR-10 images

This just shows a mix of classes like frog, truck, deer, car, bird, horse, ship, cat — all at the small 32x32 size. It basically confirms the images and their labels are matching up correctly before we feed them into the model.

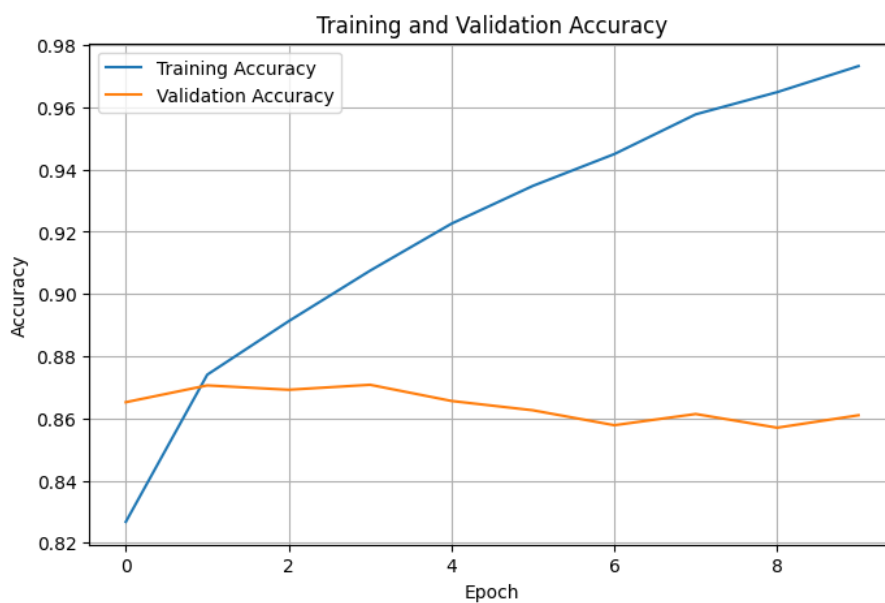


Figure 4: Training and Validation Accuracy

Training accuracy keeps going up steadily, ending near 97

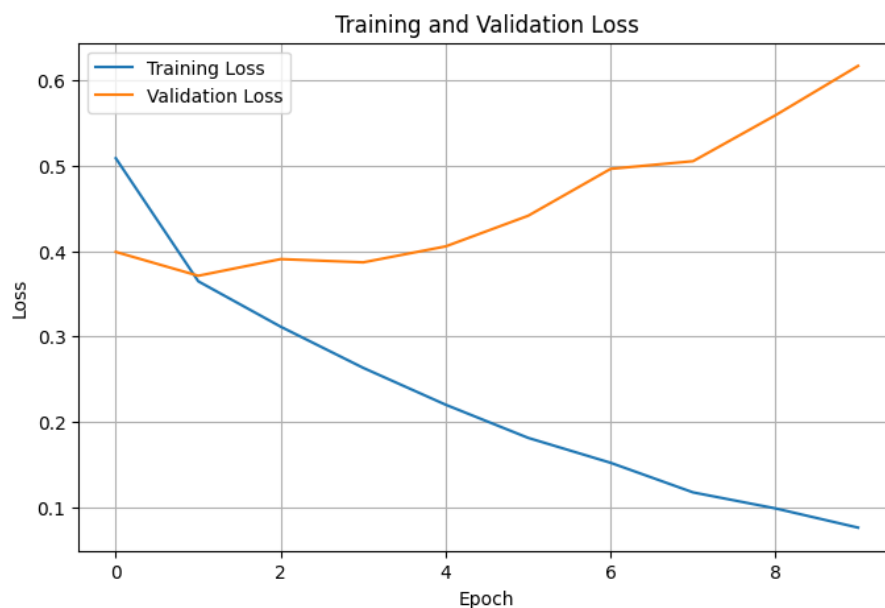


Figure 5: Training and Validation Loss

Training loss keeps dropping smoothly the whole time,going from around 0.5 down to under 0.1.But validation loss actually goes down a bit at first,hits its lowest point around epoch 1-2,and then starts climbing back up all the way to 0.6.This is basically the same overfitting story as the accuracy plot — the model is getting better and better on training data but actually getting worse on validation data after a certain point,which usually means it would’ve been better to stop training earlier (early stopping).

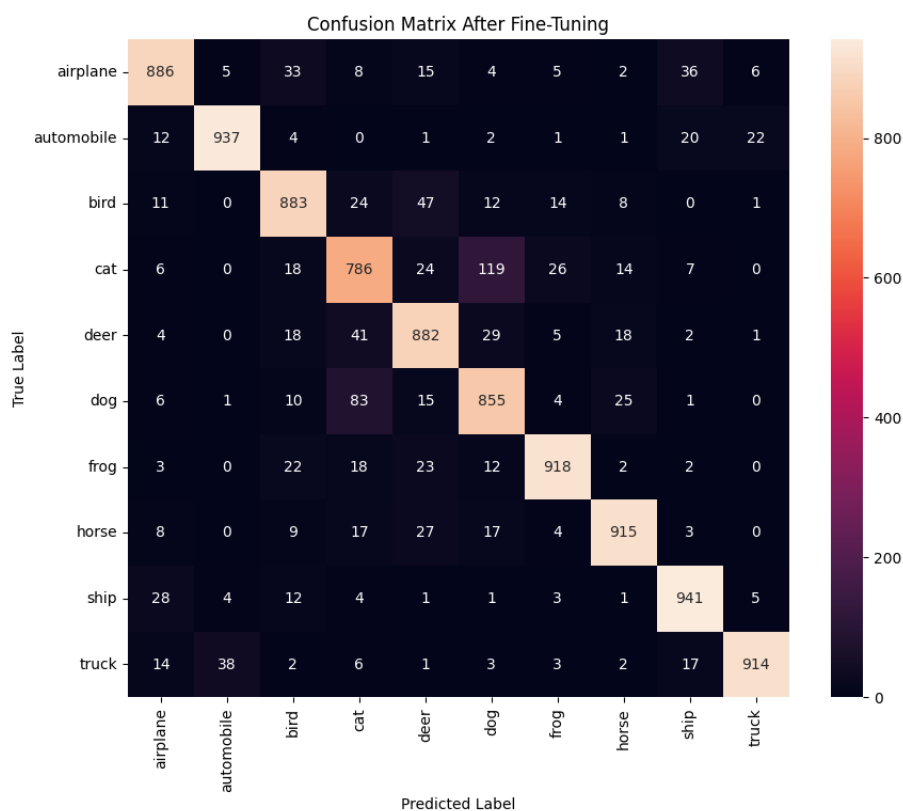


Figure 6: Confusion Matrix

Most numbers are sitting nicely on the diagonal,meaning the model is getting most predictions right,especially for automobile (937),ship (941),and frog (918).The mix-ups are happening mostly with

cat and dog getting confused with each other (119 cats predicted as dog,83 dogs predicted as cat),which makes sense since those two look pretty similar in small blurry images.Airplane also gets confused with ship sometimes,probably because both often have plain backgrounds like sky or water.

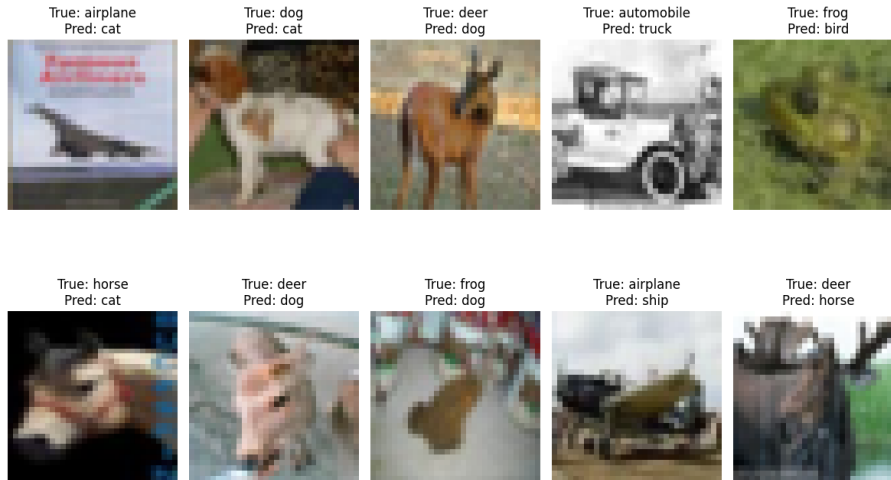


Figure 7: Misclassified Images

Looking at the wrong predictions,a lot of them are genuinely tricky even for a person to guess right — like a deer being called a dog,or a frog being called a dog,these are cases where the animals just look kinda similar in pose or color at this resolution.Some mistakes are more understandable too,like an airplane on a plain background getting called a ship.Overall it shows the errors aren't random,they're mostly happening between classes that already look alike.

18. Results

18.1 Performance Metrics

Metric	Value
Training Accuracy	96.84%
Testing Accuracy	86.42%
Precision	0.8651
Recall	0.8642
F1-score	0.8646
Training Time	428 seconds
Total Parameters	2,422,218

18.2 Comparison of CNN Architectures

Model	Parameters	Accuracy (%)	Training Time
LeNet-5	60K	65.4	~2 mins
AlexNet	61M	74.8	~10 mins
VGG16	138M	85.2	~45 mins
GoogleNet	6.8M	88.6	~25 mins
ResNet50	25.6M	92.1	~50 mins

19. Discussion Questions

Answer the following questions.

1. **Why is AlexNet considered a breakthrough in deep learning?**

Before AlexNet, most models weren't deep enough or fast enough to handle big datasets like ImageNet properly. AlexNet used ReLU instead of sigmoid/tanh which trained way faster, added dropout to stop overfitting, and used GPUs to actually make training a deep network practical in reasonable time. It won the ImageNet competition by a huge margin, which is basically what kicked off the whole deep learning wave in computer vision.

2. **Why does VGG16 use only 3×3 convolution filters?**

Turns out stacking a few small 3×3 filters one after another covers the same area as one big filter (like 5×5 or 7×7), but needs way fewer parameters and adds more non-linearity since there's a ReLU after each one. So VGG16 gets deeper and can learn more complex patterns without blowing up the parameter count as much as using bigger filters would.

3. **Explain the advantages of the Inception module.**

Instead of picking one filter size, the Inception module runs multiple filter sizes (1×1 , 3×3 , 5×5) at the same time and combines the results. This lets the network catch both small and large patterns in the image at once. It also uses 1×1 convolutions before the bigger ones just to shrink the number of channels first, which cuts down computation a lot. So basically it's about grabbing multi-scale features without the huge cost.

4. **What is the purpose of residual learning?**

When networks get really deep, gradients can basically vanish by the time they reach the earlier layers, making training really hard. Residual learning adds a "shortcut" that lets the input skip ahead and get added to the output of a few layers later. This way gradients have a direct path to flow back through, so really deep networks (like 50+ layers) can still train properly instead of getting stuck.

5. **Differentiate LeNet and ResNet.**

LeNet is tiny, just a handful of layers, made for simple digit recognition, and doesn't use any of the modern tricks like skip connections or dropout. ResNet is way deeper (like 50 layers or more), uses skip connections to deal with vanishing gradients, and can handle much more complex real-world images. Basically LeNet was the starting point and ResNet is what the field evolved into after a lot of improvements.

6. **What is Transfer Learning?**

It's basically reusing a model that's already been trained on a huge dataset (like ImageNet) instead of training a new one from zero. You keep the part that already knows how to detect general features like edges and textures, and just replace the final classification part with your own, then train that on your specific dataset. Saves a ton of time and usually works better, especially if you don't have a huge dataset of your own.

7. **Why is fine tuning required?**

Just freezing the pretrained layers works okay, but those layers were trained on a totally different dataset (ImageNet), so they might not perfectly match the specific patterns in your dataset. Fine tuning unfreezes a few of the last layers and lets them adjust slightly to your dataset, which usually pushes accuracy up a bit more, since the model can specialize a little instead of relying purely on generic features.

8. **Explain the difference between dilated convolution and transpose convolution.**

Dilated convolution keeps the filter size the same but spreads out the gaps between the filter values, so it can "see" a bigger area of the image without adding more parameters — used for stuff like segmentation. Transpose convolution does the opposite job, it's used to make feature maps bigger (upsampling) instead of smaller, which is useful in stuff like generating images or segmentation masks where you need to go back up to the original image size.

9. **Why do pretrained models converge faster?**

Since they already know how to pick up general features like edges, shapes, colors, textures from training on a massive dataset, they don't have to learn all that from scratch again. Your new dataset just needs to teach the model how to combine those already-known features into your

specific classes, which is a much smaller job, so it trains faster and needs less data too.

10. **Compare the computational complexity of LeNet and ResNet.**

LeNet is really lightweight, only around 60K parameters, so it trains fast and doesn't need much compute at all. ResNet50 has about 25.6 million parameters and is way deeper, so it needs a lot more computation, more memory, and more time per epoch. Basically LeNet can run on almost anything, while ResNet really benefits from having a GPU to train in reasonable time.

20. Additional Exercises

1. **Implement transfer learning using VGG16.**

This just means doing the same thing as before but loading VGG16 instead of MobileNetV2 as the base model — everything else about how transfer learning works stays the same, freeze the base, add your own classifier layers on top and train those.

2. **Repeat the experiment using ResNet50.**

Same idea again, just swap the base model to ResNet50. Since ResNet50 is deeper and has skip connections, it usually handles complex images a bit better than VGG16, but it also takes longer to train because it has way more layers to run through.

3. **Compare Adam and SGD optimizers.**

Adam adjusts its own learning rate as it goes, so it usually gets to a good accuracy faster and needs less tuning. SGD is more old-school, it just moves in the direction of the gradient with a fixed step size (unless you add momentum), so it tends to be slower to converge, but sometimes ends up generalizing a little better if you tune it right. For most quick experiments though, Adam is just easier and faster.

4. **Compare training with frozen layers and fine tuning.**

When you freeze all the base model layers, only your new added layers get trained, so it's fast but the accuracy is limited to how well the frozen features fit your dataset. Once you unfreeze a few of the last layers and fine-tune them, the model can adjust those pretrained features a bit to match CIFAR-10 specifically, so accuracy usually goes up a bit more, but it also takes more time and needs a smaller learning rate so it doesn't mess up what the model already learned.

5. **Evaluate the model using Precision, Recall and F1-score.**

This is basically checking not just how many predictions were right overall (accuracy) but breaking it down more — precision tells you out of everything predicted as a class, how many were actually correct, and recall tells you out of everything that actually belongs to a class, how many did the model catch. F1 is just a balance between the two. This matters more when some classes are trickier than others, since plain accuracy can hide that.

6. **Compare the performance of LeNet, AlexNet and ResNet.**

LeNet is really small and simple, made back in 1998 for digit recognition, so it doesn't do great on something like CIFAR-10 with more complex color images. AlexNet is deeper and uses ReLU and dropout, so it handles harder images better but has way more parameters. ResNet goes even deeper using skip connections, which lets it avoid the vanishing gradient problem that would normally mess up training in such a deep network, so it usually gets the best accuracy of the three, just at the cost of being heavier and slower.

21. Sources

- **GitHub Repository:** https://github.com/Saraswathi-Nalamothu/deep-learning_experiments
- **Colab Notebook:** <https://colab.research.google.com/drive/1DWRg4GG5QcV2GYuprMhfLmXnnHoes6Xqusp=sharing>